

Stay/ Design System

Stay/ Design System

Hostel & Housing Booking App — Full Reference (Spec + Code)

This document is the complete, single-source reference for the Stay/ design system: design tokens, rationale, and the full Expo + React Native implementation. It is meant to be read together so that brand rules and code stay in sync.

1. Overview

- **Platform / stack:** Expo + React Native (mobile-first; iOS & Android)
- **Headers font:** Inter (@expo-google-fonts/inter)
- **Body font:** Open Sans (@expo-google-fonts/open-sans)
- **Brand color:** Primary green #16a34a , paired with white and soft green tints
- **Layout approach:** mobile-first, 4-column grid on phones (16px margin, 12px gutter), scaling to 8 columns (tablet) and 12 columns (desktop)
- **Spacing base unit:** 4px
- **Minimum touch target:** 48px height for all interactive controls

2. Color tokens

Token	Hex	Usage
green900	#0E3B23	Headings, nav active text
green700	#16a34a	Primary brand — buttons, links, prices
green600	#15803d	Pressed / hover state
green500	#22c55e	Icons, secondary accents
green100	#dcfce7	Chips, tinted backgrounds
green50	#f0fdf4	Page background tint
white	#FFFFFF	Primary surface
ink	#102016	Primary text
inkSoft	#5B6B61	Secondary text
line	#DCE8E0	Borders / dividers
danger	#C2402F	Errors only — used sparingly

3. Typography scale (mobile-first; phone widths < 600px)

Role	Font	Weight	Size / Line height	Usage
Display	Inter	800	28 / 34	Hero headlines
H1	Inter	700	24 / 30	Screen titles
H2	Inter	700	20 / 26	Section titles
H3	Inter	600	17 / 22	Card titles
Body Large	Open Sans	400	16 / 24	Primary reading text
Body Medium	Open Sans	400	14 / 20	Secondary text, list items
Body Small	Open Sans	400	12 / 16	Fine print, metadata
Caption	Open Sans	700	11, uppercase, tracked	Eyebrows / labels

At tablet width (≥768px): Display 34, H1 28, H2 22, H3 18; body sizes stay the same.

4. Spacing & grid

Spacing scale (4px base): 4, 8, 12, 16, 20, 24, 32, 40, 48px.

Breakpoint	Columns	Margin	Gutter
< 600px (phone)	4	16px	12px
600–959px (tablet)	8	24px	16px
≥ 960px (desktop)	12	32px	24px

Radii: sm 8px, md 14px, lg 20px, pill 999px.

5. Components

- **Text** — bound to the type scale above; headers default to green900, body defaults to ink.
- **Button** — variants: `primary` (filled green700), `secondary` (green100 fill), `outline` (green700 border), `ghost` (text only). Minimum height 48px. Defaults to full width on mobile.
- **Chip** — filter pill; `active` state fills green700, `outline` state uses a neutral border for unselected filters.
- **Badge** — status pill for bookings: `success` (green), `pending` (amber), `error` (red). Red reserved strictly for errors/cancellations.
- **ListingCard** — the core booking unit: image with a “Verified host” badge and save/heart button, name, location, price per night, and rating.

6. Setup

```
npx create-expo-app stay-app
cd stay-app
npx expo install @expo-google-fonts/inter @expo-google-fonts/open-sans expo-font
```

Copy the `theme/` and `components/` folders below into the project root, then wire up `App.tsx` (or merge its font-loading logic into an existing root component).

7. Design rules / rationale (for consistency)

1. Never hardcode a color or font size outside of `theme/` — always reference `theme.colors`, `theme.type`, `theme.spacing`.
2. Red (`danger`) is reserved for true error states only — never decorative.
3. All interactive elements (buttons, chips, list rows) must have a minimum 48px touch target.
4. Build every screen mobile-first; only add tablet/desktop adjustments via `useWindowDimensions()` referencing `grid.tablet` / `grid.desktop`, never by redesigning the component.
5. Button hierarchy per screen: one `primary` action maximum; everything else is `secondary`, `outline`, or `ghost`.

8. Full source code

The following are the complete, current contents of every file in the design system package. Treat these as the single source of truth for implementation.

theme/colors.ts

```
export const colors = {
  green900: '#0E3B23', // headings, nav active text
  green700: '#16a34a', // primary brand – buttons, links, prices
  green600: '#15803d', // pressed/hover state
  green500: '#22c55e', // icons, secondary accents
  green100: '#dcfce7', // chips, tinted backgrounds
  green50: '#f0fdf4', // page background tint

  white: '#FFFFFF',
  ink: '#102016', // primary text
  inkSoft: '#5B6B61', // secondary text
  line: '#DCE8E0', // borders/dividers

  danger: '#C2402F', // errors only – use sparingly
  warning: '#9A6A18',
  warningBg: '#FCF1DD',
  dangerBg: '#F8E5E2',
} as const;

export type ColorToken = keyof typeof colors;
```

theme/spacing.ts

```
export const spacing = {
  sp1: 4,
  sp2: 8,
  sp3: 12,
  sp4: 16,
  sp5: 20,
  sp6: 24,
  sp8: 32,
  sp10: 40,
  sp12: 48,
} as const;

export const radius = {
  sm: 8,
  md: 14,
  lg: 20,
  pill: 999,
} as const;

// 4-column mobile grid: 16px margin, 12px gutter.
// Use with a flex/grid helper or a library like react-native-flex-grid if needed.
export const grid = {
  mobile: { columns: 4, margin: spacing.sp4, gutter: spacing.sp3 },
  tablet: { columns: 8, margin: spacing.sp6, gutter: spacing.sp4 },
  desktop: { columns: 12, margin: spacing.sp8, gutter: spacing.sp6 },
};
```

theme/typography.ts

```
// Fonts: Inter for headers, Open Sans for body.
// Install:
//   npx expo install @expo-google-fonts/inter @expo-google-fonts/open-sans expo-font expo-splash-screen

import {
  useFonts,
  Inter_600SemiBold,
  Inter_700Bold,
  Inter_800ExtraBold,
} from '@expo-google-fonts/inter';
import {
  OpenSans_400Regular,
  OpenSans_500Medium,
  OpenSans_600SemiBold,
  OpenSans_700Bold,
} from '@expo-google-fonts/open-sans';

export const fontFamily = {
  headSemiBold: 'Inter_600SemiBold',
  headBold: 'Inter_700Bold',
  headExtraBold: 'Inter_800ExtraBold',
  bodyRegular: 'OpenSans_400Regular',
  bodyMedium: 'OpenSans_500Medium',
  bodySemiBold: 'OpenSans_600SemiBold',
  bodyBold: 'OpenSans_700Bold',
} as const;

// Call this once at the root of the app (e.g. App.tsx) and gate rendering
// behind `fontsLoaded` so text never flashes in a system font.
export function useAppFonts() {
  return useFonts({
    Inter_600SemiBold,
    Inter_700Bold,
    Inter_800ExtraBold,
    OpenSans_400Regular,
    OpenSans_500Medium,
    OpenSans_600SemiBold,
    OpenSans_700Bold,
  });
}

// Mobile-first type scale. Sizes are for phone widths (<600px);
// bump via a useWindowDimensions check for tablet (see typography.md note).
export const type = {
  display: { fontFamily: fontFamily.headExtraBold, fontSize: 28, lineHeight: 34 },
  h1: { fontFamily: fontFamily.headBold, fontSize: 24, lineHeight: 30 },
  h2: { fontFamily: fontFamily.headBold, fontSize: 20, lineHeight: 26 },
  h3: { fontFamily: fontFamily.headSemiBold, fontSize: 17, lineHeight: 22 },
  bodyLg: { fontFamily: fontFamily.bodyRegular, fontSize: 16, lineHeight: 24 },
  bodyMd: { fontFamily: fontFamily.bodyRegular, fontSize: 14, lineHeight: 20 },
  bodySm: { fontFamily: fontFamily.bodyRegular, fontSize: 12, lineHeight: 16 },
```

```
caption: { fontFamily: fontFamily.bodyBold,      fontSize: 11, lineHeight: 14, letterSpacing: 0.5, textTransform: 'uppercase' as const },
} as const;
```

theme/index.ts

```
import { colors } from './colors';
import { spacing, radius, grid } from './spacing';
import { type, fontFamily, useAppFonts } from './typography';

export const theme = { colors, spacing, radius, grid, type, fontFamily };
export { useAppFonts };
export type Theme = typeof theme;
```

components/Text.tsx

```
import React from 'react';
import { Text as RNText, TextProps, StyleSheet } from 'react-native';
import { theme } from '../theme';

type Variant = keyof typeof theme.type;

interface Props extends TextProps {
  variant?: Variant;
  color?: string;
}

export function Text({ variant = 'bodyMd', color, style, ...rest }: Props) {
  return (
    <RNText
      style={[
        styles.base,
        theme.type[variant],
        { color: color ?? defaultColorFor(variant) },
        style,
      ]}
      {...rest}
    />
  );
}

function defaultColorFor(variant: Variant) {
  if (variant === 'display' || variant === 'h1' || variant === 'h2' || variant === 'h3') {
    return theme.colors.green900;
  }
  if (variant === 'caption') return theme.colors.green700;
  return theme.colors.ink;
}

const styles = StyleSheet.create({
  base: { includeFontPadding: false },
});
```

components/Button.tsx

```
import React from 'react';
import { Pressable, StyleSheet, ViewStyle, StyleProp } from 'react-native';
import { theme } from '../theme';
import { Text } from './Text';

type Variant = 'primary' | 'secondary' | 'outline' | 'ghost';

interface Props {
  label: string;
  onPress?: () => void;
  variant?: Variant;
  fullWidth?: boolean;
  disabled?: boolean;
  style?: StyleProp<ViewStyle>;
}

export function Button({
  label,
  onPress,
```

```

variant = 'primary',
fullWidth = true,
disabled = false,
style,
}): Props) {
  return (
    <Pressable
      onPress={onPress}
      disabled={disabled}
      style={({ pressed }) => [
        styles.base,
        variantStyles[variant],
        fullWidth && styles.fullWidth,
        pressed && { opacity: 0.85, transform: [{ scale: 0.98 }] },
        disabled && styles.disabled,
        style,
      ]}
    >
      <Text
        variant="h3"
        color={variant === 'primary' ? theme.colors.white : theme.colors.green700}
        style={{ fontFamily: theme.fontFamily.headSemiBold }}
      >
        {label}
      </Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  base: {
    minHeight: 48, // touch target floor for one-handed mobile use
    borderRadius: theme.radius.md,
    paddingVertical: 14,
    paddingHorizontal: 20,
    alignItems: 'center',
    justifyContent: 'center',
  },
  fullWidth: { width: '100%' },
  disabled: { opacity: 0.4 },
});

const variantStyles: Record<Variant, ViewStyle> = {
  primary: {
    backgroundColor: theme.colors.green700,
    shadowColor: theme.colors.green900,
    shadowOpacity: 0.15,
    shadowRadius: 8,
    shadowOffset: { width: 0, height: 2 },
    elevation: 2,
  },
  secondary: { backgroundColor: theme.colors.green100 },
  outline: {
    backgroundColor: 'transparent',
    borderWidth: 1.5,
    borderColor: theme.colors.green700,
  },
  ghost: { backgroundColor: 'transparent' },
};

```

components/Chip.tsx

```

import React from 'react';
import { Pressable, View, StyleSheet } from 'react-native';
import { theme } from '../theme';
import { Text } from './Text';

interface ChipProps {
  label: string;
  active?: boolean;
  outline?: boolean;
  onPress?: () => void;
}

export function Chip({ label, active, outline, onPress }: ChipProps) {
  return (
    <Pressable
      onPress={onPress}
      style={[
        styles.chip,
        active && styles.chipActive,
        outline && !active && styles.chipOutline,
      ]}
    >

```

```

        <Text
          variant="bodySm"
          style={{ fontFamily: theme.fontFamily.bodySemiBold }}
          color={active ? theme.colors.white : outline ? theme.colors.inkSoft : theme.colors.green700}
        >
          {label}
        </Text>
      </Pressable>
    );
  }

  type BadgeKind = 'success' | 'pending' | 'error';

  export function Badge({ label, kind = 'success' }: { label: string; kind?: BadgeKind }) {
    const map = {
      success: { bg: theme.colors.green100, fg: theme.colors.green700 },
      pending: { bg: theme.colors.warningBg, fg: theme.colors.warning },
      error: { bg: theme.colors.dangerBg, fg: theme.colors.danger },
    } as const;
    const { bg, fg } = map[kind];
    return (
      <View style={[styles.badge, { backgroundColor: bg }]}>
        <Text variant="bodySm" color={fg} style={{ fontFamily: theme.fontFamily.headBold }}>
          {label}
        </Text>
      </View>
    );
  }

  const styles = StyleSheet.create({
    chip: {
      backgroundColor: theme.colors.green100,
      paddingVertical: 7,
      paddingHorizontal: 14,
      borderRadius: theme.radius.pill,
    },
    chipActive: { backgroundColor: theme.colors.green700 },
    chipOutline: { backgroundColor: 'transparent', borderWidth: 1, borderColor: theme.colors.line },
    badge: {
      paddingVertical: 5,
      paddingHorizontal: 12,
      borderRadius: theme.radius.pill,
      alignSelf: 'flex-start',
    },
  });
});

```

components/ListingCard.tsx

```

import React from 'react';
import { View, Image, Pressable, StyleSheet } from 'react-native';
import { theme } from '../theme';
import { Text } from './Text';

interface Props {
  imageUrl: string;
  name: string;
  location: string;
  pricePerNight: number;
  currency?: string;
  rating: number;
  reviewCount: number;
  verified?: boolean;
  saved?: boolean;
  onPress?: () => void;
  onToggleSave?: () => void;
}

export function ListingCard({
  imageUrl,
  name,
  location,
  pricePerNight,
  currency = '$',
  rating,
  reviewCount,
  verified,
  saved,
  onPress,
  onToggleSave,
}: Props) {
  return (
    <Pressable onPress={onPress} style={styles.card}>
      <View style={styles.imageWrap}>
        <Image source={{ uri: imageUrl }} style={styles.image} />

```

```

        {verified && (
          <View style={styles.verifiedBadge}>
            <Text variant="bodySm" style={{ fontFamily: theme.fontFamily.headBold }} color={theme.colors.green700}>
              Verified host
            </Text>
          </View>
        )}
        <Pressable style={styles.favButton} onPress={onToggleSave}>
          <Text>{saved ? '♥' : '♡'}</Text>
        </Pressable>
      </View>

      <View style={styles.body}>
        <Text variant="h3">{name}</Text>
        <Text variant="bodySm" color={theme.colors.inkSoft} style={{ marginTop: 2 }}>
          {location}
        </Text>

        <View style={styles.metaRow}>
          <Text variant="h3" color={theme.colors.green700}>
            {currency}{pricePerNight}
          </Text>
          <Text variant="bodyMd" style={{ fontFamily: theme.fontFamily.bodySemiBold }}>
            ★ {rating.toFixed(1)} <Text variant="bodySm" color={theme.colors.inkSoft}>({reviewCount})</Text>
          </Text>
        </View>
      </View>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: theme.colors.white,
    borderRadius: theme.radius.lg,
    overflow: 'hidden',
    borderWidth: 1,
    borderColor: theme.colors.line,
    marginBottom: theme.spacing.sp4,
    shadowColor: theme.colors.green900,
    shadowOpacity: 0.06,
    shadowRadius: 8,
    shadowOffset: { width: 0, height: 2 },
    elevation: 1,
  },
  imageWrap: { height: 160, backgroundColor: theme.colors.green100 },
  image: { width: '100%', height: '100%' },
  verifiedBadge: {
    position: 'absolute',
    top: 12,
    left: 12,
    backgroundColor: 'rgba(255,255,255,0.95)',
    paddingVertical: 5,
    paddingHorizontal: 10,
    borderRadius: theme.radius.pill,
  },
  favButton: {
    position: 'absolute',
    top: 12,
    right: 12,
    width: 32,
    height: 32,
    borderRadius: 16,
    backgroundColor: 'rgba(255,255,255,0.95)',
    alignItems: 'center',
    justifyContent: 'center',
  },
  body: { padding: theme.spacing.sp4 },
  metaRow: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    marginTop: theme.spacing.sp3,
  },
});

```

App.tsx

```

import React from 'react';
import { SafeAreaView, ScrollView, View, StyleSheet } from 'react-native';
import { theme, useAppFonts } from './theme';
import { Text } from './components/Text';
import { Button } from './components/Button';

```

```
import { Chip } from './components/Chip';
import { Badge } from './components/Badge';
import { ListingCard } from './components/ListingCard';

export default function App() {
  const [fontsLoaded] = useAppFonts();
  if (!fontsLoaded) return null; // swap for an Expo splash screen in production

  return (
    <SafeAreaView style={styles.safe}>
      <ScrollView contentContainerStyle={styles.content}>
        <Text variant="h1">Hostels in Kumasi</Text>
        <Text variant="bodyMd" color={theme.colors.inkSoft} style={{ marginTop: 4, marginBottom: 16 }}>
          12 stays near KNUST
        </Text>

        <View style={styles.chipRow}>
          <Chip label="All" active />
          <Chip label="Female only" outline />
          <Chip label="Under ₵100" outline />
        </View>

        <View style={{ height: theme.spacing.sp4 }} />

        <ListingCard
          imageUrl="https://images.unsplash.com/photo-1555854877-bab0e564b8d5"
          name="Green Valley Hostel"
          location="Ayigya, Kumasi · 0.8km from KNUST"
          pricePerNight={85}
          rating={4.8}
          reviewCount={120}
          verified
        />

        <View style={styles.badgeRow}>
          <Badge label="Confirmed" kind="success" />
          <Badge label="Pending payment" kind="pending" />
        </View>

        <Button label="Book this room" onPress={() => {}} style={{ marginTop: 16 }} />
        <Button label="Save for later" variant="secondary" onPress={() => {}} style={{ marginTop: 12 }} />
      </ScrollView>
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: theme.colors.green50 },
  content: { padding: theme.spacing.sp4, paddingBottom: 48 },
  chipRow: { flexDirection: 'row', gap: 8, flexWrap: 'wrap' },
  badgeRow: { flexDirection: 'row', gap: 8, marginTop: 16 },
});
```

README.md

```
# Stay/ Design System — Expo + React Native

## Setup

```bash
npx create-expo-app stay-app
cd stay-app
npx expo install @expo-google-fonts/inter @expo-google-fonts/open-sans expo-font
```

Copy the `theme/` and `components/` folders into your Expo project root, then copy `App.tsx` over (or merge its font-loading logic into your existing root).

## Structure

```
theme/
 colors.ts — green/white palette tokens
 spacing.ts — 4px-base spacing, radius, grid breakpoints
 typography.ts — Inter (headers) + Open Sans (body) type scale, font loader
 index.ts — combined theme object
components/
 Text.tsx — Text bound to the type scale
 Button.tsx — primary / secondary / outline / ghost, 48px min height
 Chip.tsx — filter chips
 Badge.tsx — booking status pills
 ListingCard.tsx — the core hostel/room card
```

## Brand color

---

Primary green: `#16a34a` ( `theme.colors.green700` ). Full ramp in `colors.ts` from `green900` (deep, for headings) down to `green50` (page background tint).

## Mobile-first notes

---

- All components default to phone-width sizing (the `type` scale, 48px button height, 16px screen padding). Tablet/desktop adjustments should hook into `useWindowDimensions()` and switch to the `grid.tablet` / `grid.desktop` values in `spacing.ts` rather than redesigning components per breakpoint.
- Touch targets are never below 48px tall.
- `Button` defaults to `fullWidth` since mobile screens are narrow; pass `fullWidth={false}` for inline/secondary actions like “Cancel”.

## Extending

---

Add new variants (e.g. a `RoomCard` for individual rooms inside a hostel, or a `BookingSummaryCard` ) by composing `Text` , `Button` , and the existing `theme` tokens — avoid hardcoding colors or font sizes outside of `theme/` .

...

---